

ESERCIZIO PRATICO

Backend REST API con Node.js & Express

Sistema di Gestione Palestra

Informazioni esercizio

Livello	Intermedio / Avanzato
Prerequisiti	JavaScript ES6+, concetti REST, SQL, async/await
Tecnologie	Node.js 20+, Express 4, Sequelize 6, MariaDB
Endpoint richiesti	15 endpoint totali
Consegna	Codice sorgente, script SQL (creazione tabelle + dati), collection Postman

1. Obiettivi dell'esercizio

Lo studente dovrà progettare e implementare un backend REST per la gestione di una palestra, usando Node.js con Express e Sequelize come ORM. Il progetto copre la modellazione relazionale con due relazioni di tipo diverso (molti-a-uno e molti-a-molti), la gestione delle operazioni CRUD complete e la scrittura di query di interrogazione più mirate.

Al termine dell'esercizio lo studente sarà in grado di:

- Definire modelli Sequelize con relazioni belongsTo/hasMany (molti-a-uno) e belongsToMany (molti-a-molti)
- Implementare tutti gli endpoint CRUD per più risorse collegate tra loro
- Gestire correttamente le operazioni asincrone con async/await
- Scrivere query di interrogazione avanzate tramite Sequelize
- Produrre script SQL per la creazione delle tabelle e il popolamento dei dati di test
- Organizzare e documentare le chiamate API in una collection Postman

2. Struttura del database

Il database MariaDB deve contenere tre tabelle. La tabella principale è Corso. Essa è collegata a Sala con una relazione molti-a-uno (ogni corso si svolge in una sola sala, ma una sala può ospitare molti corsi) e a Iscritto con una relazione molti-a-molti (un iscritto può frequentare più corsi, un corso può avere molti iscritti).

2.1 Tabella principale — Corso

Rappresenta un corso offerto dalla palestra.

Campo	Tipo	Vincoli	Note
id	INTEGER	PK, AUTOINCREMENT	Chiave primaria autoincrementale
nome	STRING	NOT NULL	Nome del corso, es. "Spinning", "Yoga", "Functional"
descrizione	TEXT	opzionale	Descrizione del corso e degli obiettivi
livello	STRING	NOT NULL	Livello: 'base', 'intermedio', 'avanzato'
prezzoMensile	FLOAT	NOT NULL	Costo mensile in euro
postiMax	INTEGER	NOT NULL	Numero massimo di iscritti ammessi
Salald	INTEGER	FK → Sala	Relazione multi-a-uno (Sequelize: belongsTo)

2.2 Tabella secondaria — Sala (relazione Multi-a-Uno con Corso)

Rappresenta una sala fisica della palestra in cui si svolgono i corsi. Una sala può ospitare molti corsi; ogni corso si svolge in una sola sala.

In Sequelize: Sala.hasMany(Corso) e Corso.belongsTo(Sala).

Campo	Tipo	Vincoli	Note
id	INTEGER	PK, AUTOINCREMENT	Chiave primaria autoincrementale
nome	STRING	UNIQUE, NOT NULL	Es. "Sala Grande", "Sala Pesi", "Sala Corsi 1"
piano	INTEGER	NOT NULL	Piano dell'edificio in cui si trova la sala
capienza	INTEGER	NOT NULL	Numero massimo di persone che la sala può ospitare

2.3 Tabella secondaria — Iscritto (relazione Multi-a-Molti con Corso)

Rappresenta un iscritto alla palestra. Un iscritto può frequentare più corsi; un corso può avere molti iscritti. La relazione è gestita da una tabella associativa Corsoscritto (Corsold, Iscrittold).

In Sequelize: Corso.belongsToMany(Iscritto, { through: 'Corsoscritto' }) e Iscritto.belongsToMany(Corso, { through: 'Corsoscritto' }).

Campo	Tipo	Vincoli	Note
id	INTEGER	PK, AUTOINCREMENT	Chiave primaria autoincrementale
nome	STRING	NOT NULL	Nome dell'iscritto
cognome	STRING	NOT NULL	Cognome dell'iscritto

email	STRING	UNIQUE, NOT NULL	Indirizzo email dell'iscritto
dataNascita	DATEONLY	NOT NULL	Data di nascita

Nota sul diagramma ER

Prima di scrivere codice, disegnare il diagramma ER delle tre tabelle e identificare esplicitamente: chiavi primarie, chiavi esterne, cardinalità di ogni relazione e i metodi Sequelize corrispondenti (belongsTo/hasMany per la relazione con Sala, belongsToMany per la relazione con Iscritto).

3. Tecnologie e struttura del progetto

3.1 Stack tecnologico richiesto

- Node.js 20 LTS o superiore
- Express 4 — framework web per la creazione di REST API
- Sequelize 6 — ORM per Node.js compatibile con MariaDB, MySQL, PostgreSQL
- mariadb — driver nativo Node.js per MariaDB (alternativa: mysql2)
- dotenv — gestione delle variabili di ambiente per le credenziali del database

3.2 Struttura consigliata del progetto

```
palestra/
├── app.js                # entry point Express
├── config/
│   └── database.js      # connessione Sequelize + MariaDB
├── models/
│   ├── index.js         # carica modelli e associazioni
│   ├── Corso.js         # modello Sequelize Corso
│   ├── Sala.js          # modello Sequelize Sala
│   └── Iscritto.js      # modello Sequelize Iscritto
├── routes/
│   ├── corsi.js         # router endpoint corsi
│   ├── sale.js          # router endpoint sale
│   └── iscritti.js      # router endpoint iscritti
├── sql/
│   ├── create_tables.sql # script di creazione tabelle
│   └── seed_data.sql     # script di popolamento dati
├── postman/
│   └── palestra.json     # collection Postman
├── .env                 # credenziali DB (non committare!)
├── package.json
└── README.md
```

4. Elenco degli endpoint richiesti

Il progetto deve esporre esattamente 15 endpoint. Ogni endpoint deve restituire codici HTTP appropriati (200, 201, 204, 400, 404) e rispondere sempre con JSON.

4.1 Endpoint — Sale (3 endpoint)

#	Metodo	URL	Descrizione	Tag
1	GET	/api/sale	Restituisce l'elenco di tutte le sale	Sala
2	POST	/api/sale	Crea una nuova sala	Sala
3	DELETE	/api/sale/:id	Elimina una sala (solo se senza corsi associati)	Sala

4.2 Endpoint — Iscritti (4 endpoint)

#	Metodo	URL	Descrizione	Tag
4	GET	/api/iscritti	Restituisce l'elenco di tutti gli iscritti	Iscritto
5	POST	/api/iscritti	Crea un nuovo iscritto	Iscritto
6	GET	/api/iscritti/:id	Dettaglio iscritto con i corsi frequentati	Iscritto
7	PUT	/api/iscritti/:id	Aggiorna i dati di un iscritto esistente	Iscritto

4.3 Endpoint — Corsi (5 endpoint CRUD)

#	Metodo	URL	Descrizione	Tag
8	GET	/api/corsi	Lista di tutti i corsi con sala e iscritti	Corso
9	POST	/api/corsi	Crea un nuovo corso (con array di id iscritti)	Corso
10	GET	/api/corsi/:id	Dettaglio completo di un singolo corso	Corso
11	PUT	/api/corsi/:id	Modifica i dati di un corso esistente	Corso
12	DELETE	/api/corsi/:id	Elimina un corso dal catalogo	Corso

4.4 Endpoint — Interrogazioni avanzate (3 endpoint)

#	Metodo	URL	Descrizione	Tag
13	GET	/api/corsi/search	Ricerca corsi per nome (query param: ?q=...)	Ricerca
14	GET	/api/corsi/disponibili	Lista dei corsi con posti ancora disponibili	Ricerca

15	GET	/api/sale/:id/corsi	Tutti i corsi che si svolgono in una sala specifica	Ricerca
----	-----	---------------------	---	---------

5. Specifiche dettagliate degli endpoint

Endpoint 9 — POST /api/corsi

Il body JSON della richiesta deve includere:

- nome, livello, prezzoMensile, postiMax (obbligatori)
- descrizione (opzionale)
- SalalId — FK verso la tabella Sala (obbligatorio)
- iscrittiIds — array di interi con gli ID degli iscritti da associare al corso (opzionale)

Prima di creare il corso, verificare che la sala con SalalId esista, altrimenti HTTP 404 con messaggio: "Sala non trovata.". Dopo aver creato il corso con `Corso.create()`, usare il metodo `Sequelize corso.setIscritti(iscrittiIds)` per stabilire le associazioni molti-a-molti. Se uno degli ID iscritto non esiste, restituire HTTP 404 con messaggio descrittivo. Risposta di successo: HTTP 201.

Endpoint 13 — GET /api/corsi/search?q=...

Deve effettuare una ricerca case-insensitive sul campo nome utilizzando l'operatore `Op.like` di Sequelize. Su MariaDB la ricerca con `LIKE` è già case-insensitive per default con collation `utf8_general_ci`. Esempio: `/api/corsi/search?q=yoga` deve restituire tutti i corsi il cui nome contiene "yoga". Se il parametro `q` non viene fornito, restituire HTTP 400 con messaggio: "Parametro di ricerca 'q' obbligatorio.".

Attenzione all'ordine delle route!

In Express, `/api/corsi/search` e `/api/corsi/disponibili` devono essere definite PRIMA di `/api/corsi/:id` nel file delle route, altrimenti Express interpreterà 'search' e 'disponibili' come valori dinamici di `:id`.

Endpoint 14 — GET /api/corsi/disponibili

Deve restituire solo i corsi il cui numero di iscritti attuali è inferiore a `postiMax`. Il conteggio degli iscritti per ciascun corso può essere ottenuto contando gli elementi dell'associazione (ad esempio con un `include` e un conteggio lato JavaScript, oppure con una subquery Sequelize).

Endpoint 3 — DELETE /api/sale/:id

Prima di eliminare la sala, verificare che non ci siano corsi associati con `Corso.count({ where: { SalalId: id } })`. Se il conteggio è maggiore di zero, restituire HTTP 400 con il messaggio: "Impossibile eliminare: esistono corsi associati a questa sala.". Risposta di successo: HTTP 204 No Content.

6. Definizione dei modelli e delle associazioni

6.1 Associazioni in models/index.js

Dopo aver importato tutti e tre i modelli, definire le associazioni nel file models/index.js:

```
// Relazione multi-a-uno: Corso → Sala
Sala.hasMany(Corso, { foreignKey: 'SalaId' });
Corso.belongsTo(Sala, { foreignKey: 'SalaId' });

// Relazione multi-a-molti: Corso ↔ Iscritto
Corso.belongsToMany(Iscritto, { through: 'CorsoIscritto' });
Iscritto.belongsToMany(Corso, { through: 'CorsoIscritto' });
```

6.2 Include nelle query

Per restituire un corso con la sala e gli iscritti associati usare l'opzione include nelle query Sequelize:

```
const corsi = await Corso.findAll({
  include: [
    { model: Sala },
    { model: Iscritto, through: { attributes: [] } }
  ]
});
```

L'opzione through: { attributes: [] } esclude dalla risposta JSON i campi interni della tabella associativa CorsosIscritto.

7. Script SQL richiesti

Nella cartella sql/ devono essere presenti due file SQL eseguibili direttamente su MariaDB.

7.1 create_tables.sql

Deve contenere le istruzioni CREATE TABLE per tutte e quattro le tabelle (Sale, Corsi, Iscritti, CorsosIscritto), con i vincoli di chiave primaria, chiave esterna e i valori di default. Usare IF NOT EXISTS per rendere lo script rieseguibile. Le tabelle devono essere create nell'ordine corretto per rispettare i vincoli di integrità referenziale: prima Sale e Iscritti, poi Corsi, infine CorsosIscritto.

7.2 seed_data.sql

Deve contenere le istruzioni INSERT INTO per popolare il database con dati di test realistici. Sono richiesti almeno:

- 4 sale con capienze e piani diversi
- 10 corsi distribuiti tra le sale, con livelli e prezzi variati
- 15 iscritti
- associazioni nella tabella CorsosIscritto tali che ogni corso abbia almeno un iscritto, alcuni corsi risultino al completo (per testare l'endpoint 14) e alcuni iscritti frequentino più corsi contemporaneamente

8. Requisiti di consegna

Il materiale da consegnare è composto da tre elementi obbligatori:

1. Codice sorgente Node.js — Cartella del progetto completa e funzionante, con file `package.json` contenente tutte le dipendenze e uno script "start" per avviare il server. Il codice deve essere privo di errori e rispettare la struttura delle cartelle indicata al punto 3.2.
2. Script SQL — I due file `create_tables.sql` e `seed_data.sql` nella cartella `sql/`. Gli script devono essere eseguibili su un server MariaDB pulito senza alcuna modifica manuale. Verificare che l'ordine delle `CREATE TABLE` rispetti le dipendenze tra le chiavi esterne.
3. Collection Postman — File `.json` importabile in Postman con tutti e 15 gli endpoint configurati. Per gli endpoint `POST` e `PUT` devono essere inclusi body di esempio con dati realistici. La collection deve essere organizzata in tre cartelle (`Sale`, `Iscritti`, `Corsi`) che rispecchino i tag degli endpoint.

Avvertenza

L'uso di soluzioni generate automaticamente da strumenti di intelligenza artificiale senza comprensione del codice prodotto è considerato disonestà accademica. Il docente si riserva di richiedere la spiegazione di qualsiasi parte del materiale consegnato.